ONLINE TICKET BOOKING SYSTEM

Project Requirements Specification

Application Type: RESTful Backend API
Technology Focus: Spring Boot, Spring MVC, Spring Data JPA, Spring Security

A student project to apply Spring concepts in a practical, layered backend application.

Version 1.0

1. Project Overview

The Online Ticket Booking System is a backend application that allows users to browse available movies, view show timings, check seat availability, and book or cancel tickets through RESTful APIs. Administrators manage movies, theatres, screens, shows, and ticket pricing. The application must follow REST API design principles, maintain data consistency, handle exceptions, and secure protected endpoints.

2. User Roles

Customer

- Register and log in to the application.
- Browse movies and search by title or genre.
- View movie details, theatres, show dates, timings, and available seats.
- Book tickets by selecting one or more seats.
- View booking details and personal booking history.
- Cancel eligible bookings according to the cancellation rules.

Admin

- Add, update, view, and delete or deactivate movies.
- Manage theatres and screens.
- Create, update, and cancel shows.
- Set ticket prices.
- View and search bookings by booking ID, customer, or show.
- View basic booking statistics, such as total bookings.

3. Functional Requirements

3.1 User Management

- Register users, authenticate login credentials, retrieve and update the current user's profile, and allow admins to view users with pagination.
- Email addresses must be unique. Validate required fields and store passwords as BCrypt hashes.
- Users must not be able to assign or change their own role through the profile API.

3.2 Movie Management

- Movie fields: ID, title, description, genre, language, duration, release date, certification, and status (ACTIVE or INACTIVE).
- Support create, list, retrieve by ID, search by title or genre, update, and delete/deactivate operations.

3.3 Theatre and Screen Management

- Theatre fields: ID, name, location, and address. Screen fields: ID, name/number, theatre ID, and total seats.
- One theatre can contain multiple screens; each screen belongs to one theatre.
- Seat numbers must be unique within a screen. A screen cannot be deleted while it has active shows or bookings.

3.4 Show Management

- Show fields: ID, movie ID, screen ID, date, start time, end time, ticket price, and status.
- A show must reference a movie and a screen. End time must be after start time.

- Shows on the same screen must not overlap. Customers can filter shows by movie, theatre, and date.
- Only active and upcoming shows can be booked.

3.5 Seat Management

- Seat fields: ID, screen ID, seat number, and seat type (for example, REGULAR or PREMIUM).
- Return available seats for a particular show. A seat's reservation status is specific to a show, not globally fixed on the screen.
- Customers must not be able to reserve a seat already booked for the same show.
- Concurrent requests must not cause the same seat to be sold twice.

3.6 Ticket Booking

- Booking fields: ID, customer ID, show ID, selected seats, total amount, booking date/time, status, and unique booking reference.
- Allow a customer to book multiple seats in one request and calculate the total from the selected seats and applicable prices.
- Reject unavailable seats. Save the booking and seat reservations in one database transaction.
- Customers can retrieve only their own booking details and history.

3.7 Booking Cancellation

- Allow cancellation only when the booking is eligible under the cancellation policy.
- A booking cannot be cancelled twice. Mark it CANCELLED and release its seats for the same show.
- Cancellation changes must be transactional so the booking and seat availability remain consistent.
- Return a clear response when cancellation is not permitted.

Payment Scope

Payment gateway integration is excluded from the first version. The system records and displays the booking amount but does not process actual payments.

4. REST API Endpoint Requirements

Students must use appropriate HTTP methods, URL mappings, request data annotations, and response status codes. The following is the initial endpoint design:

Module	Method	Endpoint
Authentication	POST	/api/v1/auth/register
Authentication	POST	/api/v1/auth/login
Users	GET	/api/v1/users/me
Users	PUT	/api/v1/users/me
Users	GET	/api/v1/users
Movies	POST	/api/v1/movies
Movies	GET	/api/v1/movies
Movies	GET	/api/v1/movies/{id}
Movies	PUT	/api/v1/movies/{id}
Movies	DELETE	/api/v1/movies/{id}
Theatres	POST	/api/v1/theatres
Theatres	GET	/api/v1/theatres

Module	Method	Endpoint
Screens	POST	/api/v1/theatres/{theatreId}/screens
Shows	POST	/api/v1/shows
Shows	GET	/api/v1/shows
Shows	GET	/api/v1/shows/{id}
Seats	GET	/api/v1/shows/{showId}/seats
Bookings	POST	/api/v1/bookings
Bookings	GET	/api/v1/bookings/{id}
Bookings	GET	/api/v1/bookings/my-bookings
Bookings	PATCH	/api/v1/bookings/{id}/cancel

Students should add endpoints needed to update shows, manage screens and seats, and retrieve booking statistics.

Use @RestController, @RequestMapping, @GetMapping, @PostMapping, @PutMapping, @PatchMapping, @DeleteMapping, @PathVariable, @RequestParam, and @RequestBody as appropriate.

Use suitable status codes, including 200 OK, 201 Created, 204 No Content, 400 Bad Request, 401 Unauthorized, 403 Forbidden, and 404 Not Found.

5. Technical Requirements — Spring Concepts

Spring Boot and Spring MVC

- Configure the Spring Boot application and REST controllers.
- Use Controller → Service → Repository layering.
- Read request data with @PathVariable, @RequestParam, and @RequestBody.
- Return responses using ResponseEntity.

Spring Data JPA and Hibernate

- Create entities and repositories, primary/foreign keys, and suitable entity relationships.
- Use @OneToMany, @ManyToOne, and @ManyToMany only where appropriate.
- Implement CRUD, derived query methods, and custom queries.
- Use @Transactional for booking/cancellation and appropriate database locking or constraints for concurrency.

Spring Security

- Implement registration, login, BCrypt password hashing, and JWT authentication.
- Configure stateless authentication and role-based authorization for CUSTOMER and ADMIN.
- Protect endpoints and prevent customers from accessing other customers' bookings.

DTO and ModelMapper

- Create request and response DTOs; do not expose entities directly in API responses.
- Use ModelMapper for entity/DTO conversion and separate create/update DTOs when appropriate.
- Never expose password hashes or other sensitive fields.

Pagination, Sorting, and Filtering

- Use Pageable and Page for paginated results.
- Support page number, page size, and sorting.

- Filter movies by genre/language, shows by movie/theatre/date, and paginate users and booking history.

API Versioning

- Use URL-based versioning, for example /api/v1/movies.
- Keep routes consistent and understand how /api/v2 could introduce changes without breaking existing clients.

Logging and Exception Handling

- Use SLF4J rather than System.out.println() for application logging.
- Use @RestControllerAdvice and @ExceptionHandler for global error handling.
- Create meaningful custom exceptions and consistent error responses.
- Never log passwords, JWTs, or other sensitive data.

6. Additional Concepts Required

Concept	Application
Bean Validation	Validate request DTOs using annotations such as @NotBlank, @NotNull, @Email, and @Positive.
Global Exception Handling	Return consistent errors for invalid input, missing records, and business rule violations.
Database Transactions	Ensure booking and cancellation operations succeed or roll back together.
Database Constraints	Enforce unique email addresses, seat identifiers, and booking references.
Concurrency Control	Prevent two customers from booking the same seat at the same time.
Configuration Management	Configure database and app settings in application.properties or application.yml.
API Documentation	Document endpoints using Swagger UI / OpenAPI.
Testing	Test service logic and REST endpoints using JUnit, Mockito, and MockMvc.
Build and Environments	Configure Maven, profiles, and application packaging.
API Error Design	Return consistent error messages, timestamps, status codes, and request paths.

Important booking rule: Checking seat availability only in Java is not enough. The database must also prevent conflicting reservations, using an appropriate locking or constraint strategy.

7. Database Requirements

Use a relational database such as MySQL or PostgreSQL. Suggested entities/tables:

- User
- Movie
- Theatre
- Screen
- Seat
- Show
- ShowSeat — availability and reservation status for each show
- Booking
- BookingSeat — seats associated with each booking

The ShowSeat entity is important because a seat belongs to a screen, but availability depends on a particular show. Reservation status must therefore be maintained per show rather than as a single permanent status on Seat. Each booking can contain multiple seats, and each reservation must belong to the relevant show.

8. Non-Functional Requirements

- Follow a layered architecture: Controller → Service → Repository.
- Require authentication for all protected endpoints.
- Restrict movie, theatre, screen, and show management to admins.
- Allow customers to access only their own bookings.
- Use appropriate HTTP status codes and validate incoming requests.
- Preserve booking consistency in database operations.
- Support debugging through logging without exposing sensitive information.
- Make APIs testable through Postman and document them through Swagger UI.
- Keep code readable, maintainable, and structured for future enhancements.

9. Expected Student Deliverables

- Complete Spring Boot source code.
- Database schema and entity relationships.
- REST API implementation with CRUD operations.
- DTOs and ModelMapper configuration.
- Spring Security and JWT authentication.
- Pagination, sorting, and filtering.
- Logging, validation, and global exception handling.
- Swagger UI documentation.
- Postman collection covering positive and negative cases.
- README with setup, database configuration, endpoints, and run instructions.

10. Project Scope and Implementation Order

Implement the project in phases so students can build on concepts progressively:

Phase 1 — Project Setup: Spring Boot, Maven, database configuration, and layered architecture.

Phase 2 — CRUD: Movie, theatre, screen, and show management with Spring Data JPA.

Phase 3 — Request and Response Design: DTOs, ModelMapper, request reading, validation, and HTTP status codes.

Phase 4 — Advanced REST APIs: API versioning, pagination, sorting, and filtering.

Phase 5 — Error Handling and Logging: Custom exceptions, global exception handling, and SLF4J.

Phase 6 — Security: Registration, login, BCrypt, JWT, and role-based authorization.

Phase 7 — Booking Workflow: Seat availability, booking, cancellation, transactions, and concurrency control.

Final Project Expectation

The final application must be a working RESTful backend, not just a collection of CRUD endpoints. A customer should be able to register, authenticate, browse movies, select a show, check seats, book tickets, view booking history, and cancel an eligible booking. An admin should be able to manage the data required for that workflow.

Payment gateway integration is outside the first version's scope.